



Facultad de Ingeniería
Universidad de Buenos Aires

TP Escalabilidad: Money Laundering Analysis

Sistemas Distribuidos

Bautista Boeri	Luciana Larrosa	Carolina Racedo
110898	110476	110550

Índice

1. Introducción	2
2. Objetivo	2
3. Dataset	2
3.1. Transacciones	2
3.2. Cuentas	3
4. Queries a resolver	3
4.1. Query 1 — Transacciones USD menores a \$50	3
4.2. Query 2 — Transacción máxima por banco	3
4.3. Query 3 — Transacciones anómalas por formato de pago	3
4.4. Query 4 — Detección del patrón Scatter-Gather	4
4.5. Query 5 — Conteo de transacciones Wire/ACH	4
5. Arquitectura	4
5.1. Vista de Casos de Uso	4
5.2. Vista Lógica	5
5.2.1. DAG	5
5.3. Vista de Procesos	7
5.3.1. Diagramas de Actividades	7
5.3.2. Diagrama de Secuencia	11
5.4. Vista de Desarrollo	13
5.4.1. Diagrama de Paquetes	13
5.5. Vista Física	13
5.5.1. Diagrama de Robustez	13
5.5.2. Diagrama de Despliegue	14
5.6. Workers y manejo del end of file	15
5.7. Workers y el manejo de múltiples entradas	17
5.8. Batches y Batch Size	18
6. División de tareas	19

1. Introducción

El blanqueo de capital, también conocido como lavado de activos, consiste en introducir en el sistema financiero legítimo bienes o dinero provenientes de actividades ilícitas, con el objetivo de disimular su origen. En el contexto de las redes digitales de pagos, esta práctica se manifiesta a través de patrones de transferencias específicos y recurrentes que buscan ofuscar los circuitos de movimiento de dinero.

2. Objetivo

El objetivo de este trabajo práctico es diseñar e implementar un **sistema distribuido** capaz de analizar un volumen masivo de transacciones bancarias en busca de anomalías y patrones asociados al lavado de activos. El sistema debe estar optimizado para entornos multicomputadoras, soportar el incremento de nodos de cómputo para escalar horizontalmente, e incorporar un middleware propio para abstraer la comunicación basada en grupos. Asimismo, debe soportar una única ejecución del procesamiento y manejar el apagado graceful ante señales SIGTERM.

3. Dataset

El sistema trabaja sobre el dataset público de IBM de transacciones financieras para detección de lavado de activos *anti-money laundering* (AML), disponible en Kaggle. El dataset consta de dos archivos principales.

3.1. Transacciones

Cada fila representa una transacción entre dos cuentas bancarias. Los campos relevantes son:

Campo	Descripción
Timestamp	Fecha y hora de la transacción (YYYY/MM/DD HH:MM)
From Bank	ID numérico del banco de origen
Account	Número de cuenta de origen
To Bank	ID numérico del banco de destino
Account.1	Número de cuenta de destino
Amount Received	Monto recibido por la cuenta destino
Receiving Currency	Moneda en que se recibe el monto
Amount Paid	Monto pagado por la cuenta de origen
Payment Currency	Moneda en que se realiza el pago
Payment Format	Formato del pago: Wire, ACH, Cheque, Bitcoin, etc.
Is Laundering	Indicador binario (0/1) de si la transacción es fraudulenta

3.2. Cuentas

Contiene información sobre las entidades bancarias y sus cuentas. Los campos son:

Campo	Descripción
Bank Name	Nombre del banco
Bank ID	Identificador numérico del banco
Account Number	Número de cuenta
Entity ID	Identificador de la entidad propietaria
Entity Name	Nombre de la entidad

4. Queries a resolver

El sistema debe calcular los siguientes resultados a partir del dataset.

4.1. Query 1 — Transacciones USD menores a \$50

Obtener la **cuenta de origen**, **cuenta de destino** y **monto** de todas las transacciones realizadas en USD cuyo monto sea inferior a 50 USD.

Campos involucrados: From Bank, Account, To Bank, Account.1, Amount Paid, Payment Currency.

4.2. Query 2 — Transacción máxima por banco

Para cada banco de origen, obtener el **nombre del banco**, la **cuenta de origen** y el **monto** correspondiente a la transacción USD de mayor valor registrada. Requiere hacer un join entre el dataset de transacciones y el de cuentas para resolver el nombre del banco a partir del Bank ID.

Campos involucrados: From Bank, Account, Amount Paid, Payment Currency (transacciones) + Bank ID, Bank Name (cuentas).

4.3. Query 3 — Transacciones anómalas por formato de pago

Obtener la **cuenta de origen** y **monto** de las transacciones USD en el período [2022-09-06, 2022-09-15] cuyo monto sea menor al 1 % del **promedio** registrado para el mismo formato de pago en el período [2022-09-01, 2022-09-05].

Las transacciones del período posterior se almacenan mientras se calcula el promedio del período base; una vez disponible, se aplica el filtro sobre las almacenadas.

Campos involucrados: From Bank, Account, Payment Format, Amount Paid, Payment Currency, Timestamp.

4.4. Query 4 — Detección del patrón Scatter-Gather

El patrón **scatter-gather** consiste en que una cuenta de origen distribuye fondos hacia múltiples cuentas intermediarias (fan-out), y estas luego concentran el dinero en una única cuenta destino distinta (fan-in), dificultando así la trazabilidad del flujo de dinero.

Esta query identifica los pares de cuentas (**origen, destino**) que cumplen dicho patrón con una sola cuenta de separación. El filtro se aplica sobre transacciones USD del período [2022-09-01, 2022-09-05], considerando únicamente cuentas de origen que hayan transferido a **al menos 5 cuentas intermedias distintas** en dicho período.

Campos involucrados: From Bank, Account, To Bank, Account.1, Payment Currency, Timestamp.

4.5. Query 5 — Conteo de transacciones Wire/ACH

Contar el total de transacciones del período [2022-09-01, 2022-09-05] con formato de pago **Wire** o **ACH** cuyo monto, **convertido a USD**, sea menor a 1 dólar.

Campos involucrados: Timestamp, Payment Format, Amount Paid, Payment Currency.

5. Arquitectura

5.1. Vista de Casos de Uso

El diagrama muestra el único actor del sistema, el **Cliente**, y su interacción principal: solicitar el análisis de transacciones. Esa acción incluye las cinco queries del sistema.

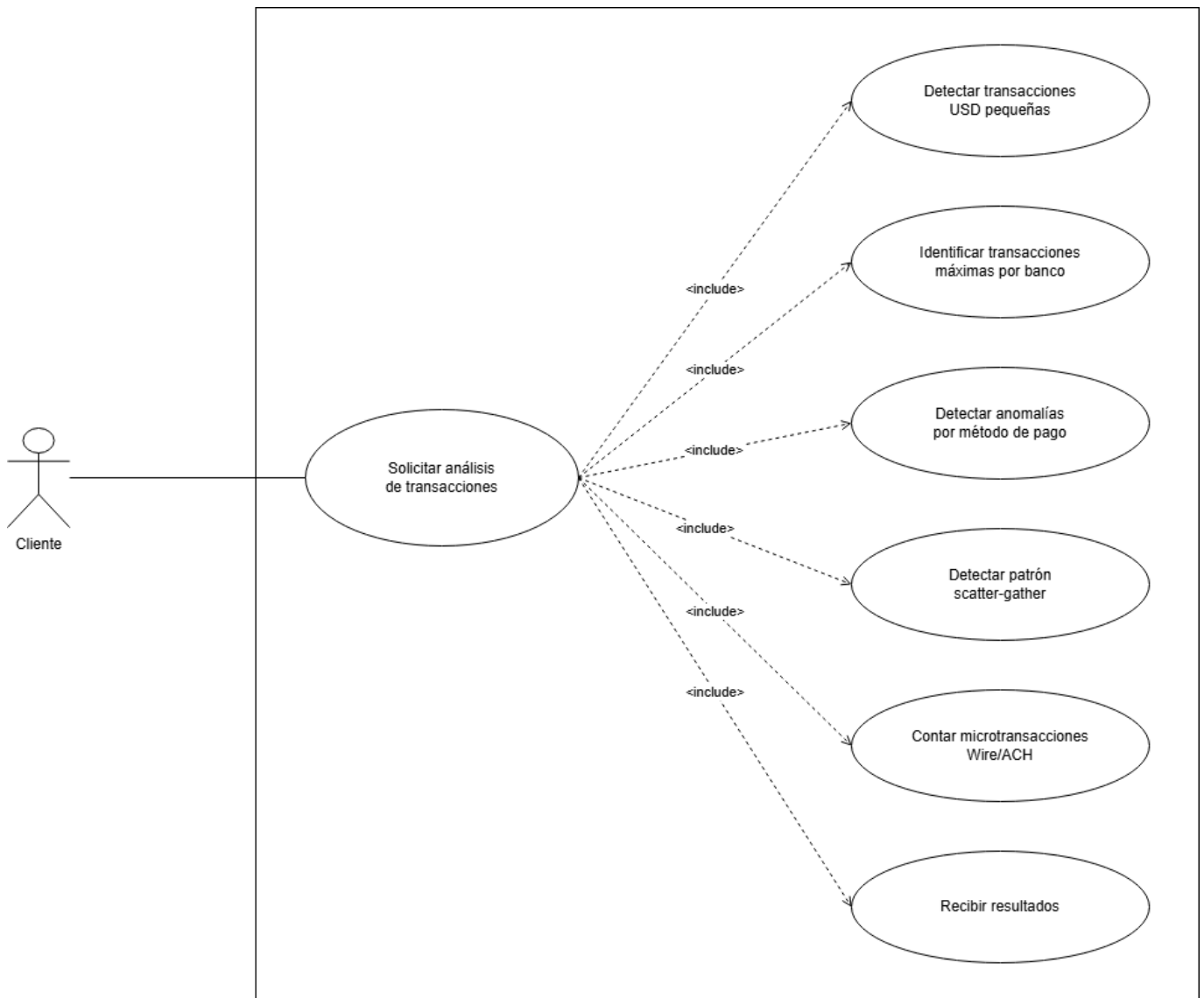


Figura 1: Diagrama de casos de uso

5.2. Vista Lógica

5.2.1. DAG

A continuación se presenta el DAG del sistema, que representa el flujo general de procesamiento de los datos. Desde **Data source** las transacciones y cuentas pasan primero por los workers **Transactions field mapper** y **Accounts field mapper** respectivamente, para normalizar los campos relevantes antes de distribuirlos al resto del sistema.

A partir de ahí, las transacciones se distribuyen por dos ramas principales: la rama **usd**, que filtra por moneda de origen, y la rama **all**, que recibe todas las transacciones independientemente de su moneda. Las cuentas, en cambio, se envían directamente al **Bank Mapper**, que las utiliza para obtener el nombre del banco en **Q2**.

Los datos van pasando por distintos nodos de procesamiento, filtrado, agregación, mapeo, entre otros;

cuyos colores en el diagrama indican el tipo de operación que realizan. Cabe destacar que algunos nodos son compartidos entre múltiples queries, como el **Date Filter**, utilizado por **Q3**, **Q4** y **Q5**. En el caso particular de **Q5**, ya no se separan las transacciones en ramas según moneda: todas pasan por el **Currency Mapper**, que se encarga de convertir los montos a USD antes de continuar con el procesamiento. Finalmente, los resultados de cada consulta son enviados al **Gateway** correspondiente.

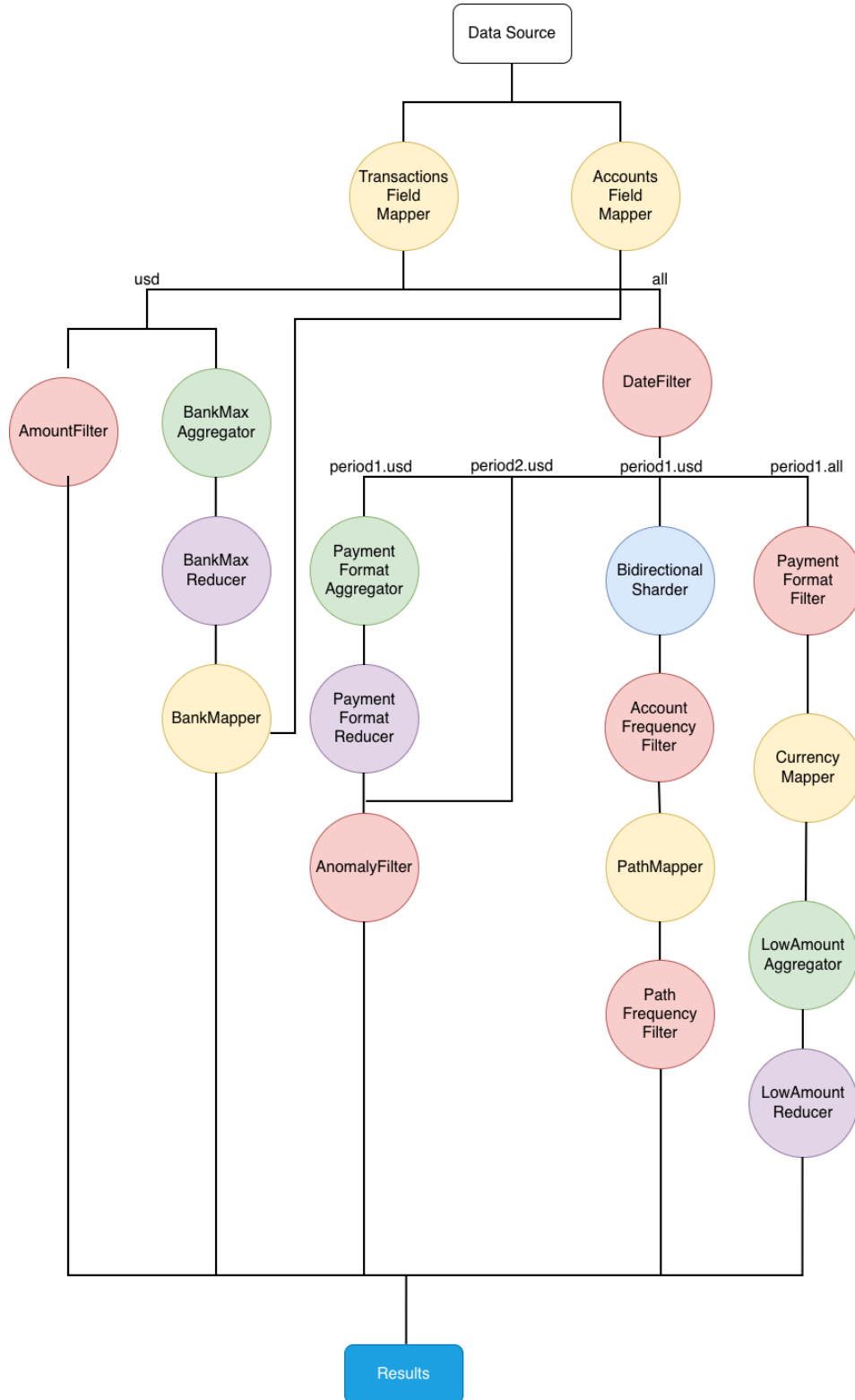


Figura 2: DAG del sistema

5.3. Vista de Procesos

5.3.1. Diagramas de Actividades

A continuación se presentan los diagramas de actividad correspondientes a cada una de las cinco consultas. Cada diagrama modela el flujo de procesamiento y consolidación de resultados para su consulta, ilustrando cómo transitan los datos a través de la topología del sistema distribuido, pasando por distintas etapas de filtrado, ruteo, transformación y agregación, hasta la consolidación y el envío de los resultados finales.

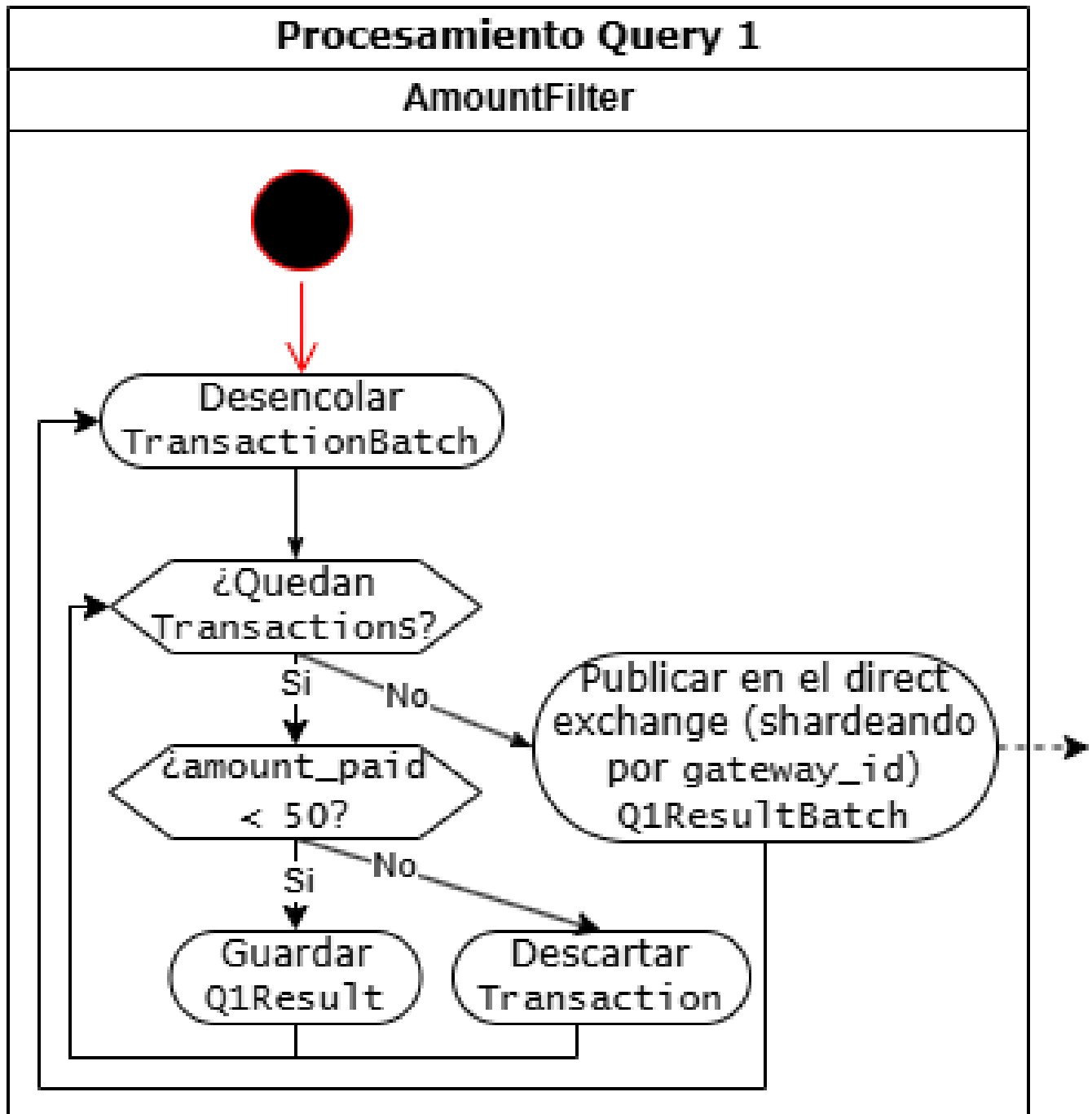


Figura 3: Diagrama de actividades — Query 1

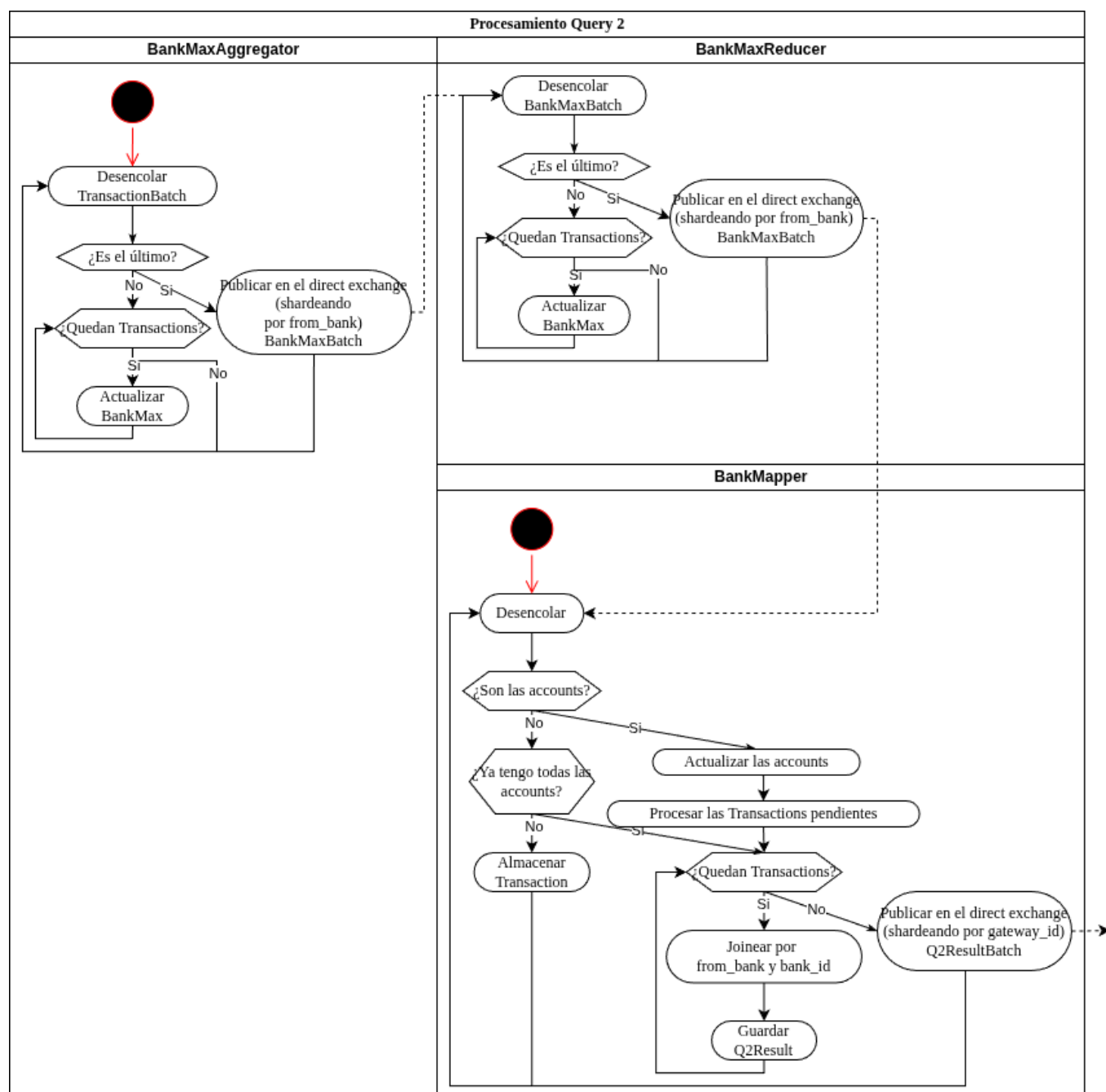


Figura 4: Diagrama de actividades — Query 2

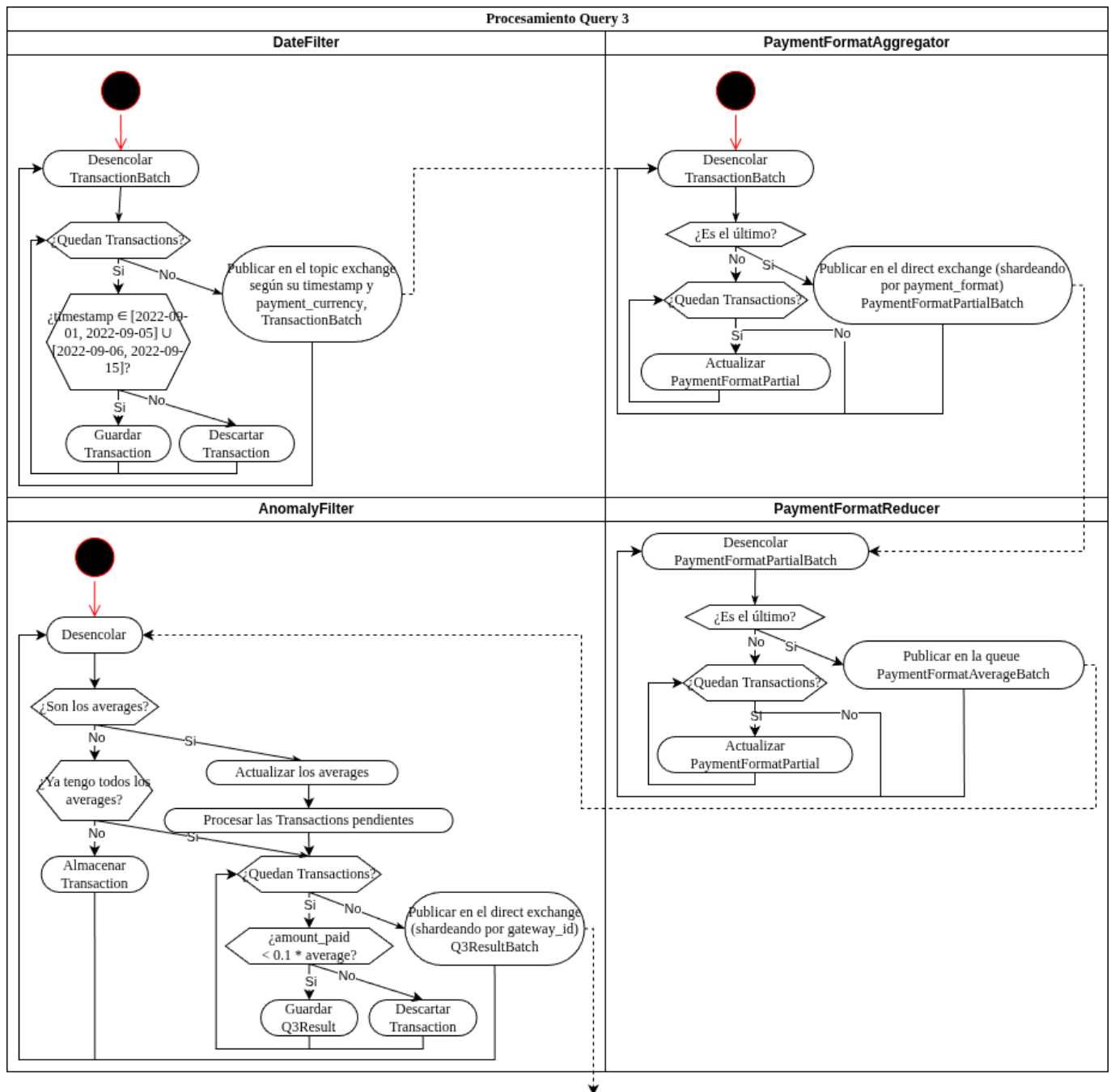


Figura 5: Diagrama de actividades — Query 3

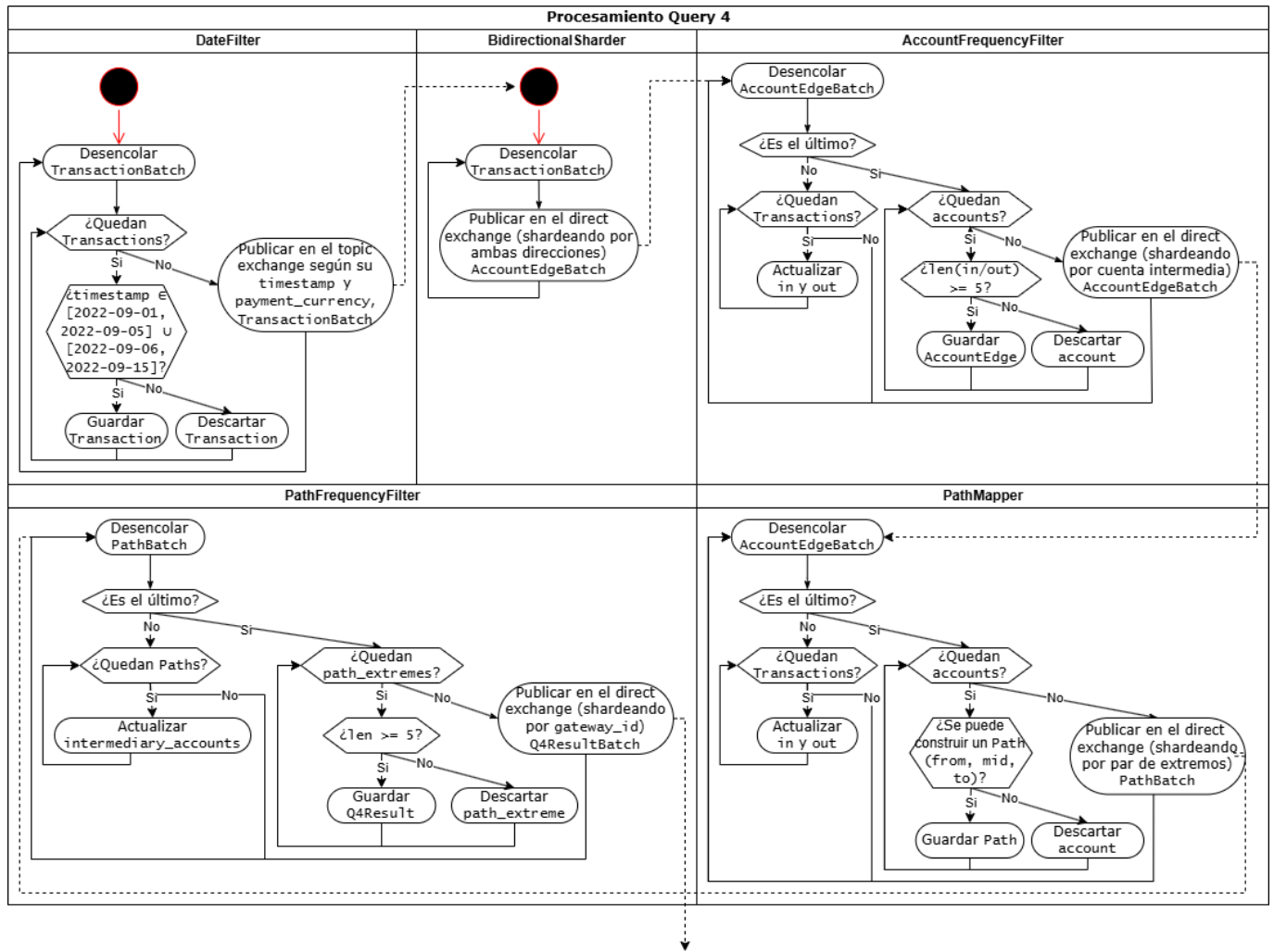


Figura 6: Diagrama de actividades — Query 4

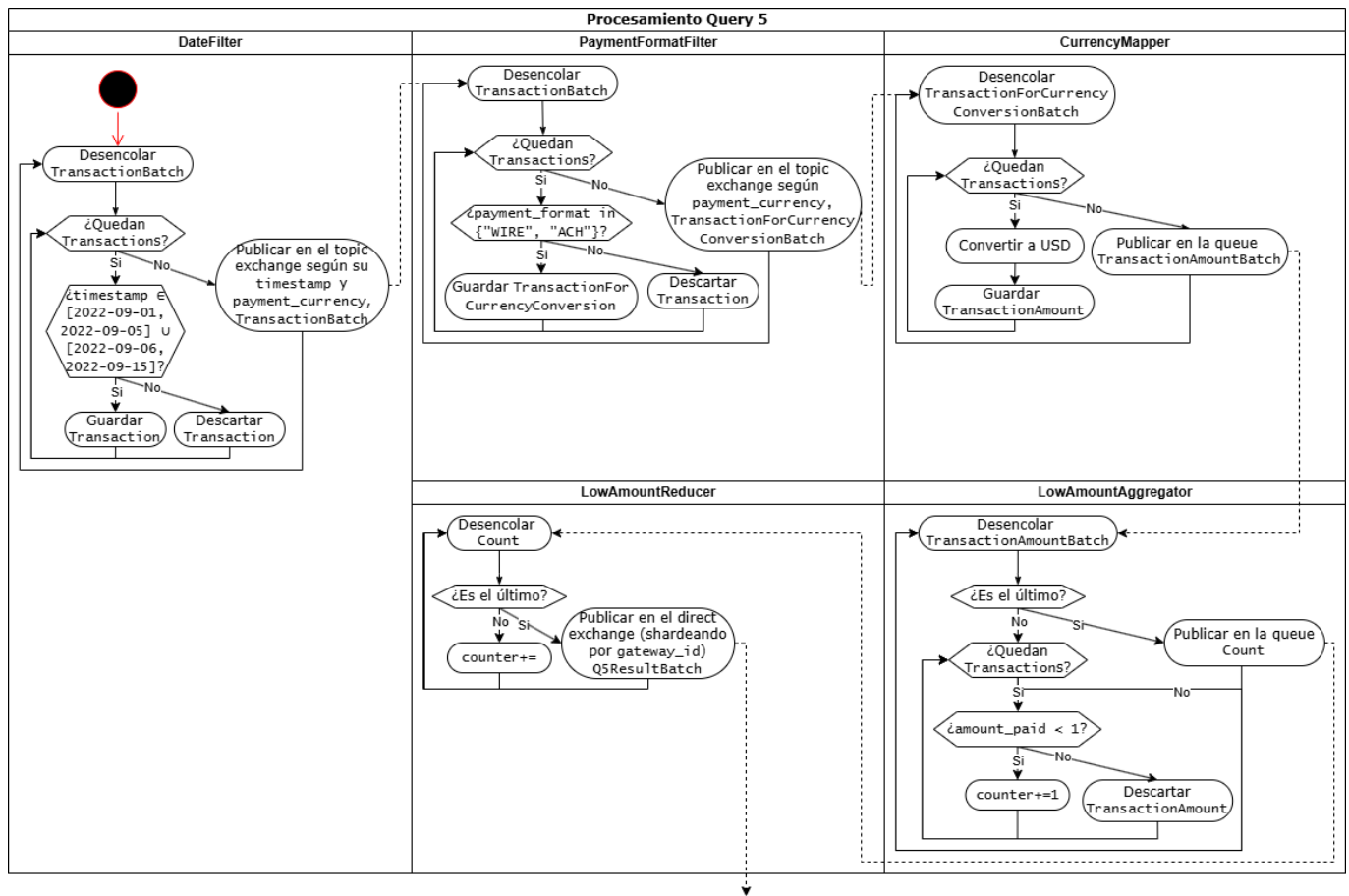


Figura 7: Diagrama de actividades — Query 5

5.3.2. Diagrama de Secuencia

El siguiente diagrama de secuencia expone la interacción general entre el cliente y los componentes de entrada y procesamiento del sistema distribuido. Se detalla el flujo de conexión inicial, donde el cliente envía una solicitud al **Proxy**, que se encarga de determinar el **Gateway** correspondiente y redirigir al cliente hacia él.

Una vez establecida la conexión con el **Gateway**, los datos son enviados en *batches*: primero las transacciones, confirmadas con un **ack** y delegadas internamente hacia los **WorkersByQuery**, hasta señalar el fin de su transmisión. Luego, de forma análoga, se envían los batches de cuentas, también delegados a los workers, finalizando con su señal de fin de transmisión correspondiente.

Una vez recibidas ambas señales, el sistema completa la etapa de procesamiento y consolidación, retornando los resultados calculados seguidos de la señal de cierre, que se propagan desde los **WorkersByQuery** a través del **Gateway** de vuelta hacia el cliente.

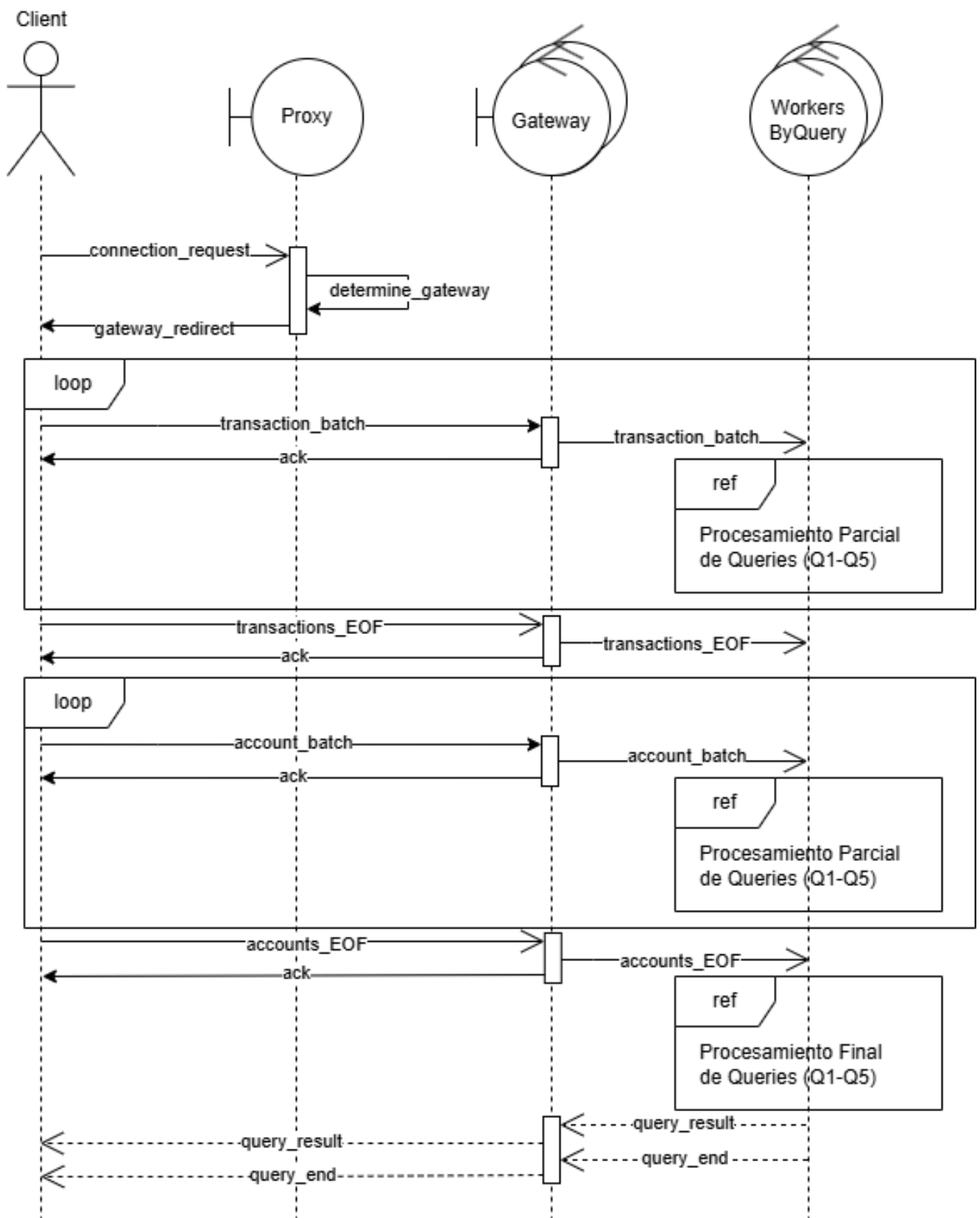


Figura 8: Diagrama de secuencia

5.4. Vista de Desarrollo

5.4.1. Diagrama de Paquetes

El diagrama de paquetes muestra la organización modular de los componentes del sistema.

El paquete **worker** representa de manera unificada a todos los nodos de procesamiento del pipeline (filtros, sharders, mappers, aggregators y reducers). Aunque cada uno tiene su lógica propia, comparten una misma estructura base (entrada desde el broker, procesamiento, salida al broker) por lo que se modelan como un único paquete para mantener el diagrama legible.

Esa estructura compartida vive en el paquete **base worker**, que agrupa las abstracciones comunes (manejo de EOFs, coordinación en anillo, ciclo de vida, etc.) sobre las que se construye cada worker concreto.

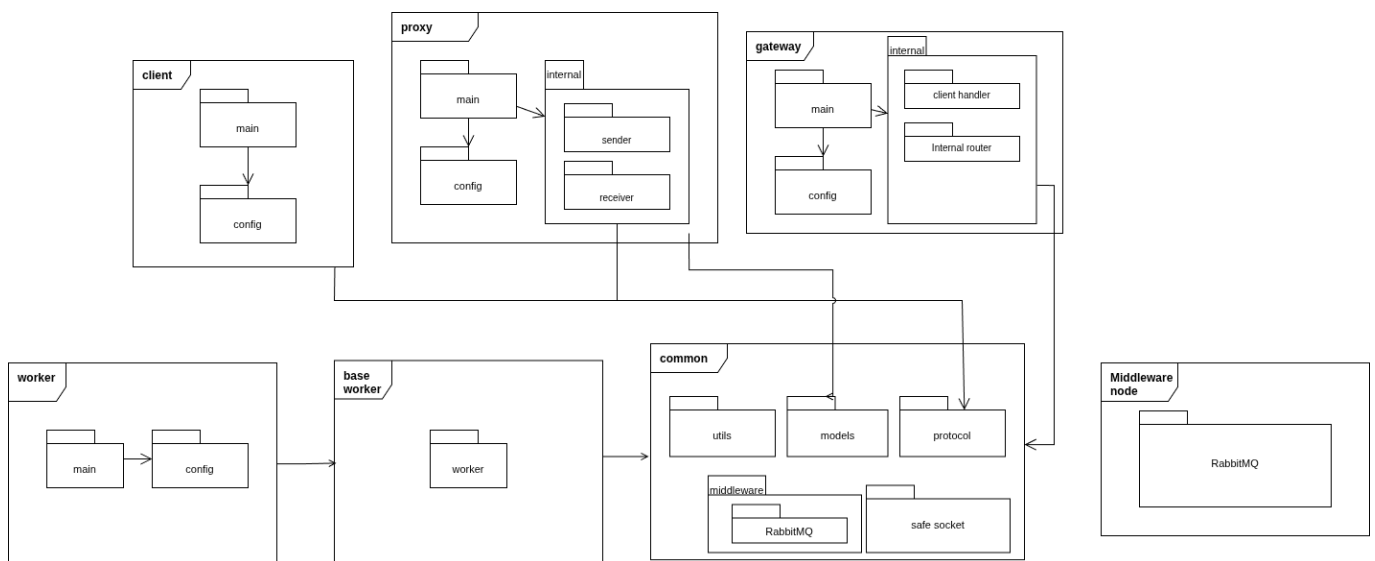


Figura 9: Diagrama de paquetes

5.5. Vista Física

5.5.1. Diagrama de Robustez

El diagrama que se encuentra a continuación muestra los componentes principales del sistema y sus interacciones. El **Cliente** se conecta primero al **Proxy**, que se encarga de indicarle a qué **Gateway** debe conectarse, ya que existen múltiples instancias disponibles. El **Proxy** cuenta con un único nodo que aplica *Round-Robin* para distribuir equitativamente los clientes entre los gateways.

Una vez que el **Cliente** obtiene el **Gateway** asignado, se conecta directamente a él y envía primero las **transacciones** en batches y luego las **cuentas**, también en batches. El **Gateway** distribuye estos datos a través de un exchange hacia los workers **Transactions Field Mapper** y **Accounts Field Mapper**, encargados de normalizar los datos antes de enviarlos al resto del sistema.

Los nodos del sistema (filtros, aggregators, mappers, entre otros) se comunican entre sí a través de

exchanges y queues, donde los exchanges permiten enrutar cada mensaje al nodo correspondiente según corresponda. Algunos nodos requieren almacenamiento temporal en disco: el **AnomalyFilter**, para retener las transacciones del período posterior mientras se calcula el promedio del período base necesario para la Query 3; y el **BankMapper**, que también persiste las transacciones mientras espera la llegada de todas las cuentas para poder comenzar el mapeo de los nombres.

A diferencia de la versión anterior, el sistema ya no cuenta con reducers como nodo final: el último worker de cada query envía los resultados directamente al **Gateway** a medida que se van generando, de forma continua, en lugar de esperar a tener el resultado consolidado. El **Gateway**, a su vez, los reenvía al **Cliente**.

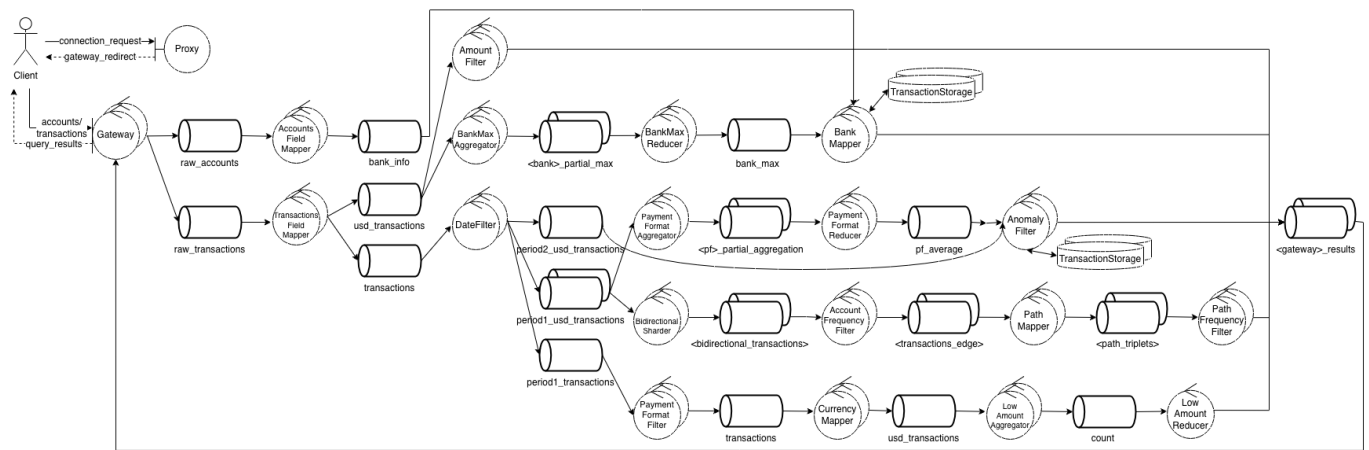


Figura 10: Diagrama de robustez

5.5.2. Diagrama de Despliegue

El diagrama de despliegue muestra cómo los distintos procesos del sistema se distribuyen en nodos de ejecución. Las líneas representan la comunicación entre nodos.

El sistema se organiza alrededor del **Broker Node** (RabbitMQ), que actúa como hub central de mensajería: todos los nodos de procesamiento se comunican entre sí exclusivamente a través de él. Las únicas conexiones por fuera del broker son las TCP entre el **Proxy Node** y el **Client PC**, y entre este último y los **Gateway Nodes**.

Los nodos de procesamiento se agrupan por rol funcional (**Filter Node**, **Sharder Node**, **Mapper Node**, **Aggregator Node**, **Reducer Node**). Cada uno de estos agrupamientos contiene múltiples implementaciones concretas con lógicas distintas (por ejemplo, el Filter Node engloba tanto el filtro por monto como el de fecha y el detector de anomalías). Se eligió agruparlos así para mantener el diagrama más simple y legible, evitando mostrar cada nodo individualmente.

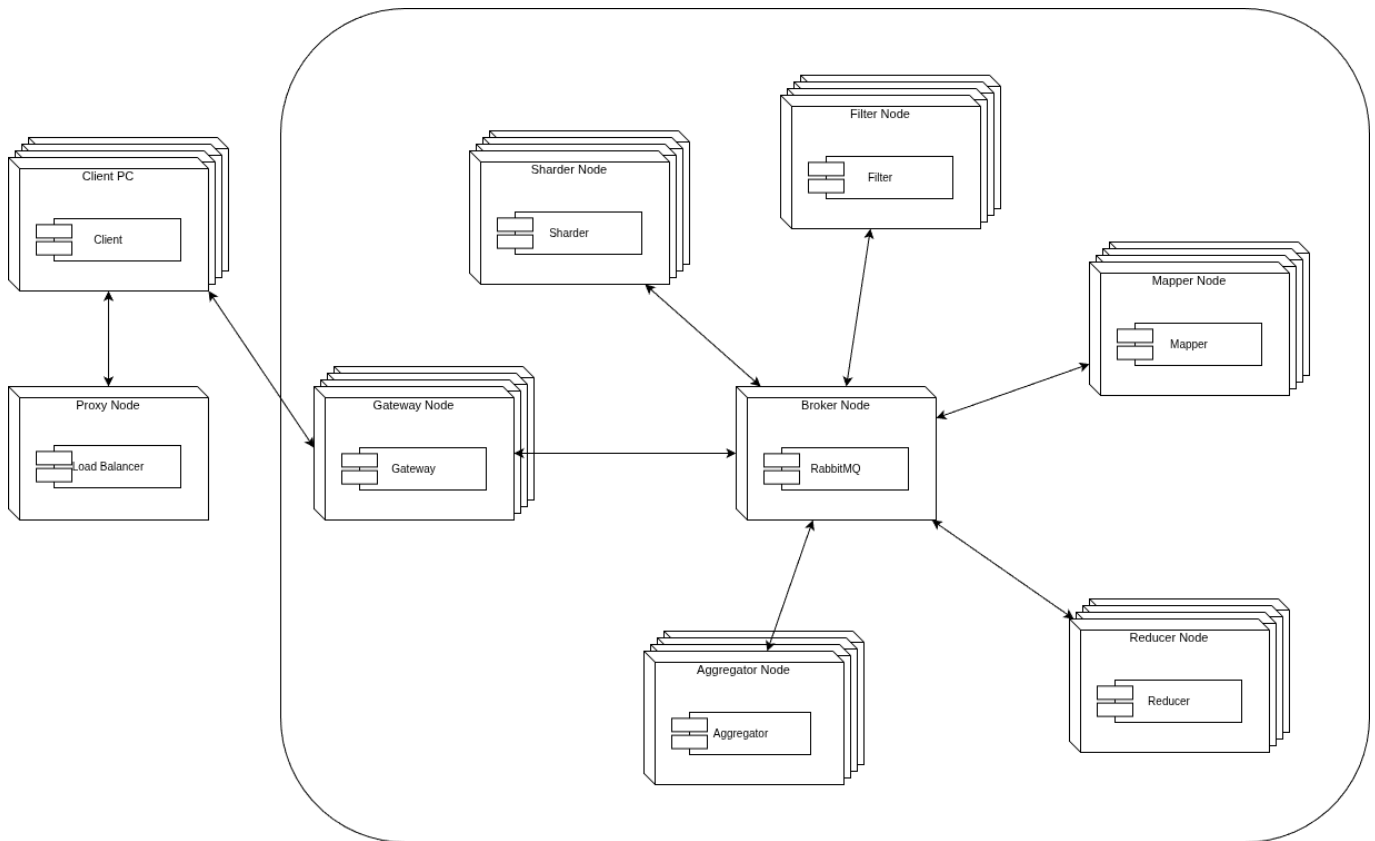


Figura 11: Diagrama de despliegue

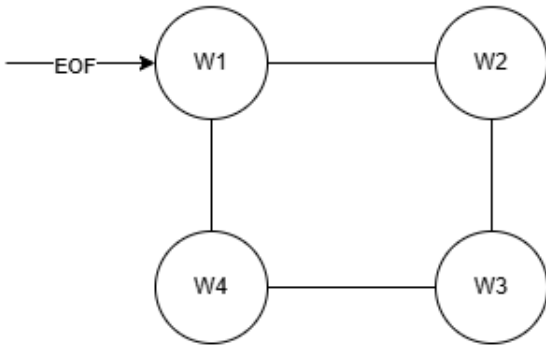
5.6. Workers y manejo del end of file

Ante un EOF, un worker puede clasificarse en una de tres categorías principales según su comportamiento:

- **StatelessWorker** es trivial, al recibir un EOF simplemente lo reenvía al siguiente stage sin modificarlo. No necesita coordinarse con nadie porque su semántica de procesamiento es 1-a-1.
- **RingCoordinatedWorker** es el núcleo del sistema distribuido. Cuando llega un EOF, el nodo no lo reenvía directamente sino que lanza un mensaje `RING_EOF` que circula por un anillo lógico de nodos. Cada nodo acumula su `processed_count` al total del `RING_EOF` antes de reenviarlo; el primero en descubrir que el acumulado alcanza el `expected_count` del EOF original se auto-designa coordinador. A partir de ahí, el `RING_EOF` da una vuelta más para que todos los nodos ejecuten `_flush_data()` (vaciar buffers, emitir resultados pendientes). Cuando el mensaje vuelve al coordinador, éste emite el EOF final al siguiente stage. Las subclases concretas difieren únicamente en cómo calculan el `expected_count` del EOF final:
 - **StatefulCoordinatedWorker**: cada nodo produce exactamente un resultado por cliente, por lo que el EOF final siempre tiene `count = ring_size`. No necesita trackear cuántos mensajes envió.

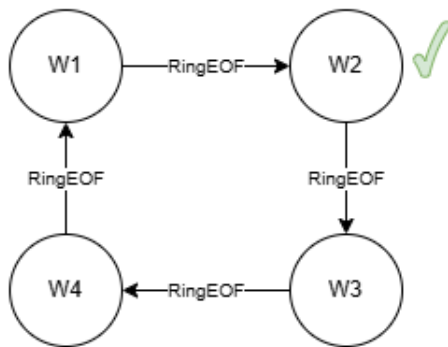
- **SentCoordinatedWorker**: la cantidad de mensajes que cada nodo envía al siguiente stage varía según si realiza batching o sharding, ya que ambos alteran la cantidad de mensajes en circulación. El EOF final debe reflejar el total real enviado, así que cada nodo acumula su `sent_count` adjuntándolo al `RING_EOF`.

Plano de Datos



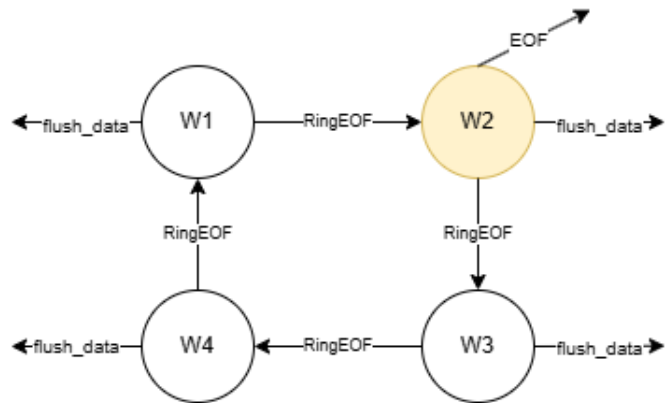
Plano de Control

Etapla 1: acumulación



N vueltas hasta que `processed == expected` → ese nodo es el coordinador

Etapla 2: flush y EOF final



*Una vuelta más para flush → el **coordinador** emite EOF con la cantidad esperada según la subclase*

Figura 12: Comportamiento del RingCoordinatedWorker

Como caso especial dentro de los workers coordinados en anillo existe `SideInputStatelessCoordinatedWorker` que incorpora una segunda fuente de datos que debe estar completamente cargada antes de poder procesar el stream principal.

En el siguiente gráfico se ilustran los tipos de workers presentes en el pipeline:

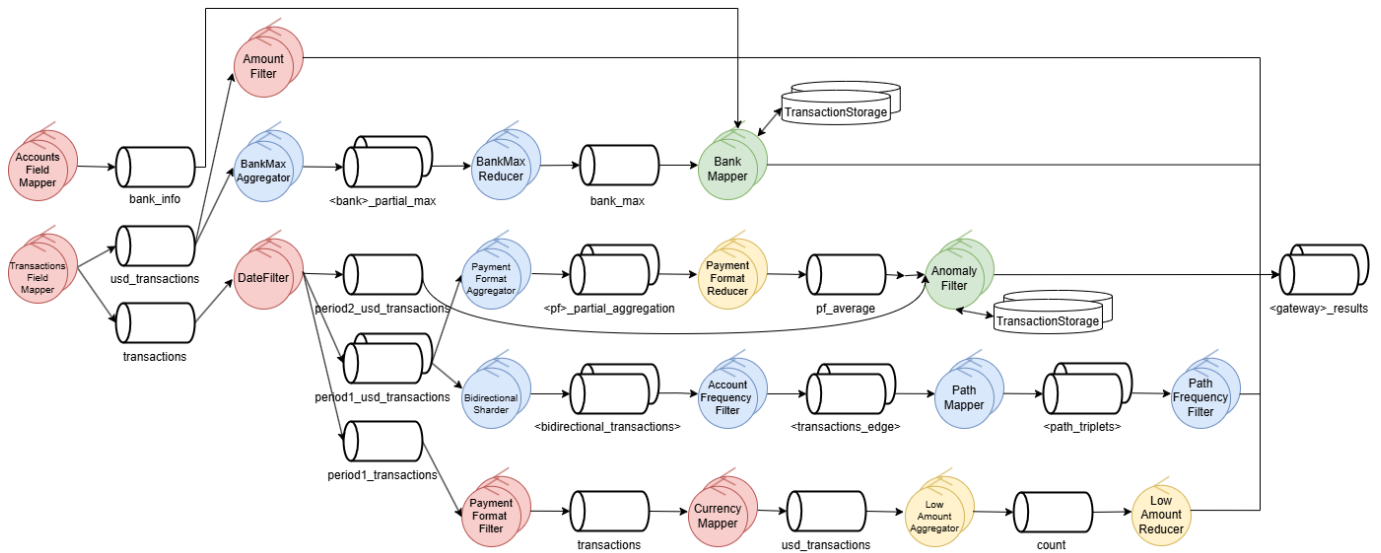


Figura 13: Workers según el manejo del EOF

El color de cada nodo indica su categoría: rojo para `StatelessWorker`, amarillo para `StatefulCoordinatedWorker`, azul para `SentCoordinatedWorker` y verde para `SideInputStatelessCoordinatedWorker`.

5.7. Workers y el manejo de múltiples entradas

Dentro de las queries 2 y 3, tenemos dos workers que van a recibir información de dos workers al mismo tiempo. De parte de la query 2, el `BankMapper` va a recibir información para mapear el nombre del banco a partir del id y también va a recibir los máximos por cuenta de cada banco. Por parte de la query 3, el `AnomalyFilter` va a recibir por un lado el promedio de transacciones de cada medio de pago y también las transacciones a filtrar.

Ambos workers están implementados sobre `SideInputStatelessCoordinatedWorker` (los nodos verdes del diagrama anterior), la variante de `RingCoordinatedWorker` mencionada arriba que suma el manejo de una segunda entrada al protocolo de anillo.

Las soluciones provistas para manejar la conexión sirven para una entrada, no para dos, por lo que se le agregó un comportamiento extra a los workers previamente mencionados. Se implementó el `SideInputTracker`, el cual va a hacer un seguimiento de la segunda entrada, en nuestros casos, los promedios y la información de los bancos. Esta información se va a recibir mediante otros exchanges y cada réplica va a recibir su propio EOF, a diferencia del ring, donde uno lo recibe y se comunican entre sí.

Es importante aclarar que el worker necesita de esta segunda entrada para poder procesar lo que recibe, por lo que si recibe entradas para procesar y no está la información completa, se guardarán en disco para ser procesadas posteriormente. Si la información ya está, se procesarán normalmente.

El spill (`BatchSpill`) almacena los batches por cliente, batch por batch, de modo de no perder el conteo que el ring necesita. Al hacer `_flush_data` se drena el archivo y se emiten los resultados pendientes por el exchange.

Como el worker necesita la segunda entrada para procesar, el comienzo de la coordinación del ring también se posterga: tanto el EOF del flujo principal como cualquier RING_EOF que llegue antes del cierre del side input quedan diferidos hasta que SideInputTracker marca el flujo como ready.

5.8. Batches y Batch Size

A lo largo de todo el pipeline, la información se transporta mediante batches. Estos pueden variar en tamaño a lo largo del mismo, en especial al encontrarse con workers que agregan información, ya que no pueden despacharla de una vez. Es por esto que se analiza el tamaño de cada batch en función de la información que contiene, respetando el tamaño máximo de frame de RabbitMQ, **128 kB**.

Para el cálculo se consideraron únicamente los mensajes del protocolo interno, ya que al viajar serializados en JSON tienen mayor overhead que otros formatos. Los máximos de chars por campo fueron obtenidos con `df[col].astype(str).str.len().max()`, representando el peor caso real del dataset.

La metadata fija de cada mensaje ocupa **129 B**, dejando **130943 B** disponibles para el payload:

```
{"type":<int>,"client_id":«uuid4>","gateway_id":«uuid4>","payload":[...]}
```

La cantidad de items por batch se calcula como $\left\lfloor \frac{\text{bytes disponibles}}{\text{bytes por item}} \right\rfloor$, ajustando por los corchetes del array y la coma del último item.

Dataclass	B/item	Items en 128 kB
RawTransaction	138	948
Transaction	221	592
RawAccount	106	1234
BankMaxPartial	87	1505
PaymentFormatPartial	103	1271
AccountEdge	162	808
Path	195	671
Q4Result	62	2112

6. División de tareas

Tarea	Integrante
Query 1	Luciana
Query 2	Bautista
Query 3	Bautista
Query 4	Carolina
Query 5	Carolina
Middleware	Bautista
Server	Luciana
Cliente	Luciana